



PRAKTIKUM PENGEMBANGAN APLIKASI WEB

MODUL 1

Lingkungan Pengembangan,
Git, dan Lalu Lintas HTTP

```
$ git init
$ git commit -m "...
$ curl -I https://...
HTTP/2 200
content-type: text/html
```

Penulis: Muhammad Fakhru Safitri, S.Kom., M.Kom.
Kode MK 52357 · 1 sks · Semester 5 · September 2026

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER · UNIVERSITAS PERTAMINA

Identitas Modul

Mata Kuliah	Praktikum Pengembangan Aplikasi Web (52357), 1 sks, semester 5
Mata Kuliah Pendamping	Pengembangan Aplikasi Web (52355), pekan 1: arsitektur aplikasi web dan protokol HTTP
Modul / Pekan	Modul 1 / Pekan 1
Sub-CPMK	Sub-CPMK 1: Mahasiswa mampu menyiapkan lingkungan pengembangan aplikasi web, mengelola kode sumber menggunakan Git dan GitHub, serta mengamati lalu lintas HTTP menggunakan DevTools, dengan minimal 80% komponen permintaan dan respons HTTP teridentifikasi dengan benar (C3, P3).
Alokasi Waktu	170 menit: Tugas Pendahuluan 20 menit (sebelum sesi), praktikum laboratorium 110 menit termasuk Tes 20 menit, dan Dokumen Teknis 40 menit (setelah sesi)
Bobot Penilaian	Tugas Pendahuluan 2%, Tes 3%, dan Dokumen Teknis 5% dari nilai akhir praktikum
Luaran	Repositori produk di GitHub dengan minimal tiga <i>commit</i> bermakna dan satu <i>pull request</i> yang telah digabungkan, serta berkas docs/praktikum/modul-01.md

A. Capaian Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. memasang dan memverifikasi Node.js 24 LTS, Git, dan Visual Studio Code sebagai lingkungan pengembangan aplikasi web;
2. membuat proyek Next.js dan menjalankannya pada server pengembangan lokal;
3. mengelola perubahan kode menggunakan Git, meliputi *commit*, *branch*, *merge*, dan penyelesaian konflik;
4. menghubungkan repositori lokal dengan GitHub, termasuk autentikasi, *push*, dan *pull request*;
5. mengidentifikasi metode, URL, kode status, dan *header* pada permintaan serta respons HTTP menggunakan DevTools dan curl.

Indikator keberhasilan: repositori produk memiliki minimal tiga *commit* bermakna dan satu *pull request* yang telah digabungkan, serta minimal 80% komponen permintaan dan respons HTTP pada lembar kerja teridentifikasi dengan benar.

B. Dasar Teori

1. Lingkungan Pengembangan Aplikasi Web

Aplikasi web modern dikembangkan dengan sejumlah perangkat yang saling melengkapi. *Runtime* menjalankan kode JavaScript di luar peramban, manajer paket mengunduh pustaka, sistem kendali versi mencatat perubahan kode, dan peramban menjalankan serta memeriksa aplikasi. Tabel 1 merangkum perangkat yang digunakan sepanjang praktikum.

Tabel 1. Perangkat lingkungan pengembangan

Perangkat	Fungsi	Versi Acuan
Node.js	<i>Runtime</i> JavaScript di luar peramban untuk menjalankan server pengembangan, proses build, dan pengujian	24 LTS
npm	Manajer paket bawaan Node.js untuk mengunduh pustaka yang tercantum pada <code>package.json</code>	Bawaan Node.js
Git	Sistem kendali versi terdistribusi	Versi stabil terbaru
Visual Studio Code	Editor kode dengan terminal terintegrasi dan dukungan ekstensi	Versi stabil terbaru
Peramban berbasis Chromium	Menjalankan aplikasi dan menyediakan DevTools untuk pemeriksaan	Versi terbaru

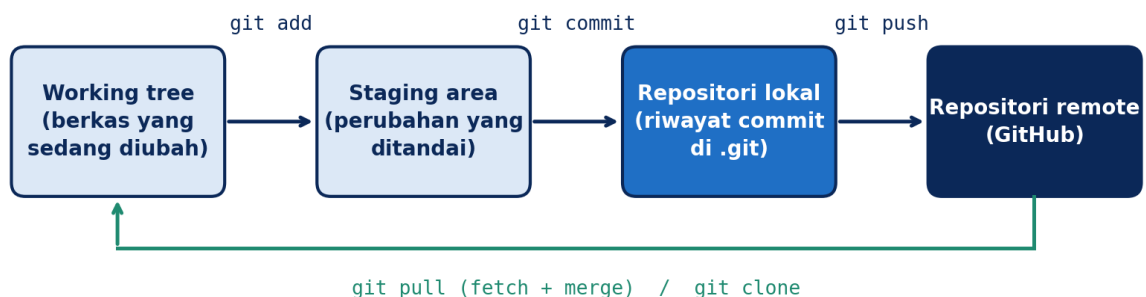
Versi LTS (*Long-Term Support*) menerima pembaruan keamanan dalam jangka panjang sehingga lebih sesuai untuk proyek sepanjang semester dibandingkan versi *Current* [9]. Next.js 16 mensyaratkan Node.js minimal versi 20.9 [5]. Seluruh modul menggunakan Node.js 24 LTS agar lingkungan setiap mahasiswa seragam dan hasil praktikum dapat direproduksi.

2. Sistem Kendali Versi dan Git

Sistem kendali versi mencatat setiap perubahan berkas sehingga pengembang dapat menelusuri riwayat, membandingkan versi, dan memulihkan kondisi sebelumnya. Git menyimpan riwayat sebagai rangkaian potret (*snapshot*) proyek yang disebut **commit**. Setiap *commit* memiliki *hash* unik, identitas penulis, waktu, dan pesan yang menjelaskan perubahan [1, Bab 1].

Git bersifat terdistribusi: setiap salinan repositori menyimpan seluruh riwayat, sehingga sebagian besar operasi dapat dilakukan tanpa koneksi jaringan. Kolaborasi dilakukan dengan bertukar *commit* melalui **repositori remote**, yang dapat berada pada server mana pun. GitHub merupakan salah satu layanan penyedia repositori *remote*. Gambar 1 menunjukkan perpindahan perubahan di antara area kerja Git.

Alur perubahan berkas dalam Git



Gambar 1. Alur perubahan berkas di antara *working tree*, *staging area*, repositori lokal, dan repositori *remote*

- **Working tree** berisi berkas proyek yang sedang diubah.

- **Staging area** (*index*) berisi perubahan yang telah ditandai dengan `git add` untuk dimasukkan ke *commit* berikutnya.
- **Repositori lokal** (folder `.git`) menyimpan seluruh riwayat *commit*.
- **Repositori remote** menyimpan salinan riwayat yang dibagikan, misalnya di GitHub.

Branch, merge, dan konflik

Branch adalah penunjuk yang bergerak ke sebuah *commit*. Dengan membuat *branch* baru, pengembang dapat mengerjakan fitur tanpa mengubah *branch* main. **Merge** menggabungkan riwayat dua *branch*. **Konflik** terjadi ketika dua *branch* mengubah baris yang sama pada berkas yang sama; Git tidak dapat menentukan versi yang benar sehingga pengembang harus menyelesaikannya secara manual [1, Bab 3].

Berkas .gitignore

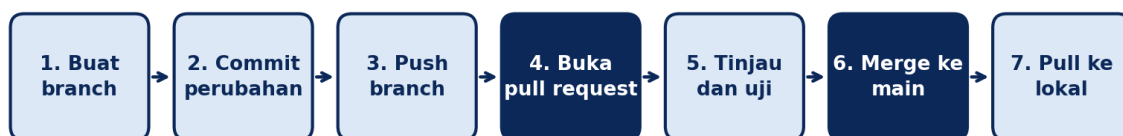
Berkas `.gitignore` berisi pola berkas yang tidak dilacak oleh Git, misalnya `node_modules/` yang dapat dipulihkan dengan `npm install` dan `.env*` yang berisi rahasia seperti kredensial basis data dan kunci API. Berkas yang telah di-*commit* tetap tersimpan dalam riwayat meskipun kemudian dicantumkan pada `.gitignore`. Oleh karena itu, rahasia tidak boleh di-*commit* sejak awal; rahasia yang terlanjur di-*commit* harus dianggap bocor dan segera diganti.

3. GitHub, Autentikasi, dan Pull Request

GitHub menyediakan repositori *remote* beserta fitur kolaborasi seperti *pull request*, peninjauan kode, dan GitHub Actions. GitHub tidak menerima kata sandi akun untuk operasi Git melalui HTTPS [6]. Autentikasi dilakukan melalui salah satu cara berikut: (a) Git Credential Manager yang terpasang bersama Git for Windows dan membuka proses masuk melalui peramban; (b) GitHub CLI dengan perintah `gh auth login`; atau (c) kunci SSH.

Pull request adalah usulan untuk menggabungkan perubahan dari sebuah *branch* ke *branch* lain, disertai ruang diskusi dan peninjauan sebelum perubahan digabungkan [7]. Alur kerja pada Gambar 2 digunakan sepanjang semester.

Alur kerja berbasis branch dan pull request (GitHub flow)



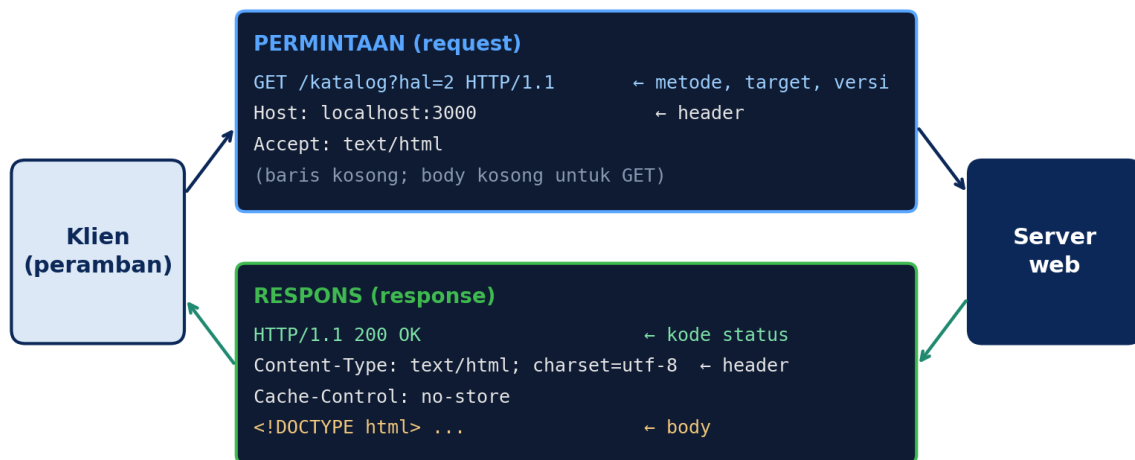
Branch main selalu berisi kode yang siap dijalankan; setiap perubahan masuk melalui pull request.

Gambar 2. Alur kerja berbasis branch dan pull request

4. Protokol HTTP

HTTP (*Hypertext Transfer Protocol*) adalah protokol aplikasi berbasis permintaan dan respons antara klien dan server [2], [3]. Klien mengirim **permintaan** yang terdiri atas metode, target (URL), *header*, dan *body* yang bersifat opsional. Server membalas dengan **respons** yang terdiri atas kode status, *header*, dan *body*. HTTP bersifat *stateless*: setiap permintaan diproses

secara independen, sehingga informasi seperti status login dipertahankan melalui mekanisme tambahan, misalnya cookie. Gambar 3 menunjukkan struktur permintaan dan respons.



Gambar 3. Struktur permintaan dan respons HTTP

Metode HTTP menyatakan tindakan yang diminta klien. Metode *safe* tidak dimaksudkan mengubah keadaan server, sedangkan metode *idempotent* menghasilkan efek yang sama meskipun dikirim berulang kali [2].

Tabel 2. Metode HTTP yang umum digunakan

Metode	Kegunaan	Safe	Idempotent
GET	Mengambil representasi sumber daya	Ya	Ya
HEAD	Sama dengan GET tanpa <i>body</i> respons	Ya	Ya
POST	Mengirim data untuk diproses, misalnya membuat data baru	Tidak	Tidak
PUT	Mengganti seluruh representasi sumber daya	Tidak	Ya
PATCH	Mengubah sebagian sumber daya	Tidak	Tidak
DELETE	Menghapus sumber daya	Tidak	Ya

Tabel 3. Kelas kode status HTTP

Kelas	Makna	Contoh
1xx	Informasional	101 Switching Protocols
2xx	Permintaan berhasil	200 OK, 201 Created, 204 No Content
3xx	Pengalihan atau penggunaan <i>cache</i>	301 Moved Permanently, 304 Not Modified
4xx	Galat pada permintaan klien	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Galat pada server	500 Internal Server Error, 503 Service Unavailable

HTTPS adalah HTTP yang dikirim melalui TLS sehingga isi komunikasi terenkripsi dan identitas server dapat diverifikasi. Lalu lintas HTTP dapat diamati melalui panel **Network** pada DevTools [4] dan melalui perintah **curl** pada terminal [10].

C. Alat dan Bahan

- Laptop dengan sistem operasi Windows 10/11, macOS, atau Linux, ruang penyimpanan kosong minimal 5 GB, dan koneksi internet.
- Akun GitHub aktif dengan autentikasi dua faktor.
- Node.js 24 LTS, Git, Visual Studio Code, serta peramban Google Chrome atau Microsoft Edge versi terbaru.
- Ekstensi Visual Studio Code: ESLint, Prettier, dan Tailwind CSS IntelliSense.
- Gagasan awal produk yang akan dikembangkan pada MK Pengembangan Aplikasi Web; gagasan dapat diubah paling lambat pekan 4.

D. Persiapan dan Instalasi

Catatan

Seluruh langkah pada bagian ini diselesaikan **sebelum sesi praktikum**, paling lambat satu hari sebelumnya. Langkah ini memerlukan pengunduhan berkas berukuran besar sehingga waktunya bergantung pada kecepatan internet dan tidak termasuk alokasi 20 menit Tugas Pendahuluan.

Pembuatan proyek pada Bagian D.4 dilakukan sebelum sesi agar pengunduhan dependensi tidak dilakukan serentak di laboratorium. Waktu sesi laboratorium digunakan sepenuhnya untuk Bagian E.

1. Node.js 24 LTS

1. Windows dan macOS: unduh installer versi LTS dari <https://nodejs.org>, kemudian jalankan dengan pilihan bawaan.
2. Linux: pasang melalui manajer versi nvm atau paket distribusi resmi sesuai petunjuk pada <https://nodejs.org>.
3. Tutup lalu buka kembali terminal, kemudian verifikasi instalasi:

Terminal

```
node -v
# v24.x.x
npm -v
# 11.x.x atau versi yang lebih baru
```

2. Git

1. Windows: unduh installer dari <https://git-scm.com/downloads>. Pertahankan pilihan bawaan, termasuk Git Credential Manager. Pada pilihan nama *branch* awal, pilih *Override the default branch name for new repositories* dan isi *main*.
2. macOS: jalankan `xcode-select --install`, atau pasang melalui Homebrew apabila Homebrew telah tersedia.
3. Linux (Debian/Ubuntu): jalankan `sudo apt install git`.

4. Atur identitas penulis *commit*. Gunakan alamat surel yang sama dengan akun GitHub.

Terminal

```
git config --global user.name "Nama Lengkap"
git config --global user.email "nim@student.universitaspertamina.ac.id"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global --list
```

Opsi `core.editor` menjadikan Visual Studio Code sebagai editor pesan *commit* sehingga mahasiswa tidak perlu menggunakan editor bawaan terminal.

3. Visual Studio Code dan Akun GitHub

1. Pasang Visual Studio Code dari <https://code.visualstudio.com>, lalu pasang ekstensi ESLint, Prettier, dan Tailwind CSS IntelliSense.
2. Buka terminal terintegrasi melalui menu *Terminal > New Terminal* atau pintasan `Ctrl + `` (backtick).
3. Buat atau masuk ke akun GitHub, verifikasi alamat surel, dan aktifkan autentikasi dua faktor melalui *Settings > Password and authentication*.

4. Pembuatan Proyek Produk

1. Buat folder kerja, misalnya `D:\praktikum-paw` (Windows) atau `~/praktikum-paw` (macOS/Linux), lalu buka folder tersebut di terminal.
2. Buat proyek Next.js. Gunakan nama produk dalam huruf kecil yang dipisahkan tanda hubung. Apabila nama produk belum ditetapkan, gunakan nama sementara; nama repositori dapat diubah kemudian.

Terminal

```
npx create-next-app@latest nama-produk
```

Saat muncul pertanyaan *Would you like to use the recommended Next.js defaults?*, pilih **Yes, use recommended defaults**. Pilihan ini mengaktifkan TypeScript, ESLint, Tailwind CSS, App Router, dan berkas `AGENTS.md` [5].

3. Masuk ke folder proyek dan jalankan server pengembangan:

Terminal

```
cd nama-produk
npm run dev
```

Buka `http://localhost:3000` pada peramban. Hentikan server dengan menekan `Ctrl + C` pada terminal.

Tabel 4. Pemeriksaan akhir persiapan

Perintah	Hasil yang Diharapkan
<code>node -v</code>	Menampilkan versi <code>v24.x.x</code>
<code>npm -v</code>	Menampilkan nomor versi npm
<code>git --version</code>	Menampilkan nomor versi Git

Perintah	Hasil yang Diharapkan
<code>git config --global --list</code>	Memuat <code>user.name</code> , <code>user.email</code> , dan <code>init.defaultbranch=main</code>
<code>npm run dev</code> di folder proyek	Halaman awal Next.js tampil pada <code>http://localhost:3000</code>

E. Langkah Praktikum

Praktikum dilaksanakan dalam lima bagian dengan pembagian waktu pada Tabel 5. Setiap bagian diakhiri titik periksa (*checkpoint*) yang diperiksa oleh asisten atau dosen sebelum melanjutkan ke bagian berikutnya.

Tabel 5. Pembagian waktu sesi laboratorium

Bagian	Kegiatan	Waktu
1	Memeriksa proyek dan struktur awal	10 menit
2	Mencatat perubahan dengan Git	20 menit
3	<i>Branch</i> , <i>merge</i> , dan penyelesaian konflik	20 menit
4	Repositori GitHub dan <i>pull request</i>	25 menit
5	Mengamati lalu lintas HTTP	15 menit
–	Tes	20 menit

Bagian 1 — Memeriksa Proyek dan Struktur Awal

1. Buka folder proyek yang telah dibuat pada Bagian D.4, jalankan `npm run dev`, dan pastikan halaman awal tampil pada `http://localhost:3000`. Mahasiswa yang belum membuat proyek segera melapor kepada asisten.
2. Buka proyek di Visual Studio Code dengan perintah `code .`, lalu pelajari struktur awal proyek pada Tabel 6.
3. Ubah teks judul pada `app/page.tsx`, simpan, dan amati bahwa halaman diperbarui secara otomatis tanpa memuat ulang server.

Tabel 6. Struktur awal proyek Next.js

Berkas/Folder	Fungsi
<code>app/</code>	Rute dan halaman aplikasi; <code>page.tsx</code> adalah halaman utama dan <code>layout.tsx</code> adalah tata letak akar
<code>public/</code>	Aset statis seperti gambar dan ikon
<code>package.json</code>	Identitas proyek, daftar dependensi, dan skrip seperti <code>dev</code> , <code>build</code> , dan <code>lint</code>
<code>package-lock.json</code>	Versi pasti setiap dependensi agar instalasi dapat direproduksi
<code>node_modules/</code>	Pustaka hasil unduhan npm; tidak di-commit
<code>.gitignore</code>	Daftar pola berkas yang tidak dilacak Git
<code>tsconfig.json</code>	Konfigurasi kompilator TypeScript
<code>AGENTS.md</code>	Panduan bagi asisten kecerdasan artifisial agar menghasilkan kode Next.js yang mutakhir

Checkpoint 1

Halaman awal Next.js tampil pada `http://localhost:3000`, perubahan pada `app/page.tsx` terlihat di peramban, dan mahasiswa dapat menjelaskan fungsi setiap berkas pada Tabel 6.

Bagian 2 — Mencatat Perubahan dengan Git

1. Periksa status repositori dan riwayat *commit*. `create-next-app` umumnya telah menginisialisasi repositori Git beserta *commit* awal. Apabila muncul pesan *not a git repository*, jalankan `git init`, `git add .`, dan `git commit -m "chore: inisialisasi proyek"`.

Terminal

```
git status
git log --oneline
```

2. Ubah isi `README.md` sehingga memuat nama produk, deskripsi singkat permasalahan yang diselesaikan, dan cara menjalankan proyek.
3. Buat berkas `.env.local` berisi `DATABASE_URL="contoh"` untuk memeriksa bahwa berkas rahasia tidak dilacak Git.
4. Jalankan `git status`. Keluaran berikut menunjukkan bahwa hanya `README.md` yang berubah, sedangkan `.env.local` tidak muncul karena tercakup pola `.env*` pada `.gitignore`.

Keluaran `git status`

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

5. Periksa isi perubahan, tandai, lalu simpan sebagai *commit*:

Terminal

```
git diff
git add README.md
git status --short
# M README.md
git commit -m "docs: tambahkan deskripsi produk pada README"
```

Pesan *commit* ditulis dalam bentuk kalimat perintah yang singkat dan diawali jenis perubahan, misalnya `feat` untuk fitur, `fix` untuk perbaikan, `docs` untuk dokumentasi, dan `chore` untuk pemeliharaan, mengikuti konvensi *Conventional Commits* [8].

6. Buat *commit* kedua untuk perubahan judul pada `app/page.tsx` dari Bagian 1, lalu tampilkan riwayat:

Terminal

```
git log --oneline
# 9c41d0a feat: ganti judul halaman utama
# 5e8782b docs: tambahkan deskripsi produk pada README
# 5b2c327 Initial commit from Create Next App
```

Perintah pada Tabel 7 digunakan untuk membatalkan perubahan sesuai kondisinya.

Tabel 7. Perintah pembatalan perubahan

Perintah	Kegunaan
<code>git restore <berkas></code>	Membuang perubahan pada <i>working tree</i> yang belum ditandai
<code>git restore --staged <berkas></code>	Mengeluarkan berkas dari <i>staging area</i> tanpa membuang perubahannya
<code>git revert <hash></code>	Membuat <i>commit</i> baru yang membalik <i>commit</i> tertentu; aman untuk riwayat yang telah dibagikan

Perhatian

Perintah `git reset --hard` menghapus perubahan yang belum di-commit secara permanen dan menulis ulang riwayat. Jangan gunakan perintah ini pada *branch* yang telah di-push ke GitHub.

Checkpoint 2

Riwayat memuat minimal dua *commit* baru dengan pesan yang bermakna, dan `.env.local` tidak muncul pada `git status`.

Bagian 3 — Branch, Merge, dan Penyelesaian Konflik

Bagian ini sengaja menimbulkan konflik agar mahasiswa terbiasa menyelesaikannya secara terkendali.

1. Buat *branch* baru, ubah kalimat deskripsi pada `README.md`, lalu buat *commit*:

Terminal

```
git switch -c latihan/konflik
# ubah kalimat deskripsi pada README.md, lalu:
git commit -am "docs: perjas deskripsi produk"
```

2. Kembali ke main, ubah **kalimat yang sama** dengan isi berbeda, lalu buat *commit*:

Terminal

```
git switch main
# ubah kalimat deskripsi yang sama dengan isi berbeda, lalu:
git commit -am "docs: ubah deskripsi produk"
```

3. Gabungkan *branch* latihan ke main. Git melaporkan konflik:

Terminal dan keluaran

```
git merge latihan/konflik
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

4. Buka README.md. Bagian yang berkonflik diapit penanda berikut:

README.md

```
<<<<<< HEAD
Deskripsi aplikasi kasir UMKM
=====
Deskripsi produk pemantau energi
>>>>>> latihan/konflik
```

Baris di antara <<<<<< HEAD dan ===== berasal dari *branch* aktif (main), sedangkan baris di antara ===== dan >>>>>> latihan/konflik berasal dari *branch* yang digabungkan. Tentukan isi akhir yang benar, hapus ketiga penanda, lalu simpan berkas. Visual Studio Code juga menyediakan pilihan *Accept Current Change*, *Accept Incoming Change*, dan *Accept Both Changes*.

5. Selesaikan *merge*, tampilkan grafik riwayat, lalu hapus *branch* latihan:

Terminal

```
git add README.md
git commit -m "merge: selesaikan konflik deskripsi produk"
git log --oneline --graph
git branch -d latihan/konflik
```

Checkpoint 3

Grafik `git log --oneline --graph` menunjukkan *commit merge*, dan README.md tidak lagi memuat penanda konflik.

Bagian 4 — Repositori GitHub dan Pull Request

1. Buat repositori baru di GitHub melalui tombol *New repository*. Gunakan nama yang sama dengan folder proyek dan pilih visibilitas *Private*. **Jangan** mencentang pilihan *Add a README file*, *.gitignore*, maupun *license*, karena riwayat lokal telah memuat berkas tersebut.
2. Siapkan autentikasi dengan salah satu cara pada Tabel 8. Cara pertama paling sederhana bagi pengguna Windows.

Tabel 8. Cara autentikasi Git ke GitHub

Cara	Langkah
Git Credential Manager (Windows)	Terpasang bersama Git for Windows. Pada <code>git push</code> pertama, jendela peramban terbuka untuk masuk ke GitHub; kredensial kemudian disimpan secara aman.

Cara	Langkah
GitHub CLI (macOS, Linux, Windows)	Pasang GitHub CLI dari https://cli.github.com (macOS: <code>brew install gh</code>). Jalankan <code>gh auth login</code> , pilih <i>GitHub.com</i> , protokol <i>HTTPS</i> , dan <i>Login with a web browser</i> . Perintah ini juga mengonfigurasi Git untuk memakai kredensial tersebut.
Kunci SSH	Buat kunci dengan <code>ssh-keygen -t ed25519 -C "email@contoh.ac.id"</code> , salin isi berkas <code>~/.ssh/id_ed25519.pub</code> ke <i>Settings > SSH and GPG keys > New SSH key</i> , lalu uji dengan <code>ssh -T git@github.com</code> . Gunakan alamat <i>remote</i> berformat <code>git@github.com:NAMA-AKUN/nama-produk.git</code> .

3. Hubungkan repositori lokal dengan GitHub, lalu kirim *branch* main:

Terminal

```
git remote add origin https://github.com/NAMA-AKUN/nama-produk.git
git push -u origin main
```

Opsi `-u` menetapkan `origin/main` sebagai *branch* hulu (*upstream*), sehingga perintah `git push` dan `git pull` berikutnya tidak memerlukan argumen tambahan.

4. Buat perubahan melalui *pull request*. Tambahkan bagian *Teknologi* dan *Cara Menjalankan* pada `README.md` di *branch* baru:

Terminal

```
git switch -c docs/readme-lengkap
# lengkapi README.md, lalu:
git commit -am "docs: lengkapi teknologi dan cara menjalankan"
git push -u origin docs/readme-lengkap
```

5. Buka halaman repositori di GitHub, klik *Compare & pull request*, lalu isi judul dan deskripsi yang menjelaskan perubahan serta cara memeriksanya. Periksa tab *Files changed*, kemudian klik *Merge pull request* dan *Delete branch*.

6. Sinkronkan repositori lokal:

Terminal

```
git switch main
git pull
git branch -d docs/readme-lengkap
```

7. Tambahkan akun GitHub dosen dan asisten sebagai kolaborator melalui *Settings > Collaborators > Add people*, sesuai daftar akun pada pengumuman e-learning. Pada repositori milik akun pribadi, GitHub memberikan hak tulis kepada setiap kolaborator; dosen dan asisten hanya menggunakan akses tersebut untuk membaca, meninjau, dan memberi komentar.

Checkpoint 4

Tab *Pull requests* menampilkan satu *pull request* berstatus *Merged*, dan riwayat lokal sama dengan riwayat di GitHub.

Bagian 5 — Mengamati Lalu Lintas HTTP

1. Jalankan `npm run dev`, buka `http://localhost:3000`, lalu buka DevTools dengan menekan F12 (Windows/Linux) atau Cmd + Option + I (macOS). Pilih panel **Network**, centang **Disable cache**, kemudian muat ulang halaman.
2. Klik permintaan pertama bertipe *document*. Pada tab **Headers**, catat *Request URL*, *Request Method*, *Status Code*, dan *Remote Address*, beserta *header* respons seperti *Content-Type* dan *Cache-Control*.
3. Buka `http://localhost:3000/halaman-tidak-ada` dan amati kode status yang diterima.
4. Buka `https://developer.mozilla.org`. Gunakan filter *Doc*, *CSS*, *JS*, dan *Img* untuk membedakan jenis sumber daya. Hapus centang *Disable cache*, muat ulang, lalu bandingkan kolom *Status* dan *Size* dengan pemuatan sebelumnya. Sumber daya dari *cache* dapat tampil sebagai 200 dengan keterangan (*memory cache*) atau (*disk cache*) tanpa permintaan ke server, atau sebagai 304 apabila peramban memvalidasi ulang salinan *cache* ke server. Catat bentuk yang muncul pada pengamatan Anda.
5. Amati permintaan dan respons melalui terminal menggunakan `curl`:

Terminal

```
curl -I http://localhost:3000
curl -I http://github.com
curl -v https://example.com
```

Opsi `-I` hanya menampilkan *header* respons, sedangkan `-v` menampilkan baris permintaan (diawali `>`) dan baris respons (diawali `<`). Karena opsi `-I` mengirim permintaan dengan metode **HEAD**, metode yang dicatat untuk pengamatan melalui `curl -I` adalah **HEAD**, bukan **GET**. Permintaan ke `http://github.com` menghasilkan kode status 301 dengan *header* *Location* yang mengarah ke alamat **HTTPS**. Pada Windows PowerShell, gunakan `curl.exe` agar yang dijalankan adalah program `curl`.

6. Catat hasil pengamatan pada lembar kerja Tabel 9, lalu salin ke Dokumen Teknis.

Tabel 9. Lembar kerja pengamatan HTTP

No	URL	Metode	Kode Status	Content-Type	Header Lain yang Diamati
1	<code>http://localhost:3000/</code>				
2	<code>http://localhost:3000/halaman-tidak-ada</code>				
3	Satu berkas CSS atau JS dari localhost				
4	<code>http://github.com</code> (curl)				
5	<code>https://developer.mozilla.org</code> (dengan <i>cache</i>)				

Checkpoint 5

Lembar kerja terisi lengkap dan mahasiswa dapat menjelaskan perbedaan kode status yang diamati, termasuk perbedaan respons dari jaringan dan dari *cache*.

F. Tugas Pendahuluan

Waktu: 20 menit, dikumpulkan melalui e-learning sebelum sesi praktikum sesuai jadwal pada e-learning. **Bobot:** 2% nilai akhir praktikum.

1. Unggah tangkapan layar keluaran `node -v`, `npm -v`, `git --version`, dan `git config --global --list`, serta halaman awal proyek pada `http://localhost:3000` dari Bagian D.4.
2. Jelaskan perbedaan *working tree*, *staging area*, dan repositori lokal pada Git.
3. Jelaskan alasan berkas `.env` tidak boleh di-commit dan tindakan yang harus dilakukan apabila berkas tersebut terlanjur di-commit.
4. Sebutkan komponen permintaan HTTP dan komponen respons HTTP.
5. Jelaskan perbedaan kode status 401 dan 403 disertai satu contoh situasi untuk masing-masing.
6. Tuliskan gagasan awal produk Anda dalam satu kalimat yang memuat permasalahan dan pengguna sasaran.

Penilaian menggunakan kunci jawaban untuk nomor 2–5 dan kelengkapan bukti untuk nomor 1 dan 6.

G. Tes

Waktu: 20 menit terakhir sesi laboratorium, dikerjakan secara individual melalui e-learning tanpa bantuan kecerdasan artifisial. **Bobot:** 3% nilai akhir praktikum.

Tes terdiri atas 10 soal yang mengukur kemampuan:

- mengidentifikasi metode, URL, kode status, dan *header* dari tangkapan panel Network DevTools;
- memilih perintah Git yang tepat untuk situasi tertentu, misalnya mengeluarkan berkas dari *staging area* atau membatalkan *commit* yang telah di-push;
- membaca penanda konflik dan menentukan hasil penggabungan.

Penilaian menggunakan kunci jawaban.

H. Dokumen Teknis

Waktu: 40 menit setelah sesi, paling lambat tiga hari setelah praktikum. **Bobot:** 5% nilai akhir praktikum.

Dokumen Teknis disimpan pada berkas `docs/praktikum/modul-01.md` di repositori produk, di-commit, lalu di-push. Tautan berkas tersebut di GitHub dikumpulkan melalui e-learning. Gunakan kerangka berikut.

`docs/praktikum/modul-01.md`

Dokumen Teknis Modul 1 – Lingkungan Pengembangan, Git, dan Lalu Lintas HTTP

Nama/NIM :
Repositori :

1. Lingkungan Pengembangan

Tabel versi sistem operasi, Node.js, npm, Git, dan Visual Studio Code.

2. Alur Kerja Git

- Keluaran git log --oneline --graph
- Tautan pull request yang telah digabungkan
- Konflik yang terjadi, cara penyelesaian, dan alasan pemilihan isi akhir

3. Pengamatan Lalu Lintas HTTP

- Lembar kerja pengamatan (Tabel 9) beserta tangkapan layar DevTools
- Keluaran curl -I dan curl -v
- Analisis: perbedaan status dan ukuran antara pemuatan dengan dan tanpa cache,
alasan metode curl -I adalah HEAD, dan alasan http://github.com dialihkan

4. Kendala dan Penyelesaian

5. Catatan Pemanfaatan AI

Alat, perintah utama, bagian yang digunakan, dan cara memverifikasinya.
Tulis "Tidak menggunakan AI" apabila tidak menggunakan AI.

Tabel 10. Rubrik Dokumen Teknis

Kriteria	Bobot	4 (Sangat Baik)	3 (Baik)	2 (Cukup)	1 (Kurang)
Ketepatan teknis	35%	Seluruh perintah, konfigurasi, dan istilah tepat	Terdapat satu kekeliruan kecil	Terdapat beberapa kekeliruan	Banyak kekeliruan mendasar
Bukti pengujian	30%	Lembar kerja lengkap dengan tangkapan layar dan keluaran terminal	Satu bukti belum lengkap	Sebagian besar bukti belum lengkap	Bukti tidak tersedia
Penjelasan keputusan teknis	20%	Analisis konflik dan kode status runtut serta beralasan	Analisis tepat tetapi kurang mendalam	Analisis bersifat deskriptif	Tidak ada analisis
Pencatatan pemanfaatan AI	15%	Lengkap, spesifik, dan disertai cara verifikasi	Lengkap tanpa cara verifikasi	Sangat umum	Tidak dicantumkan

Nilai Dokumen Teknis = $\Sigma (\text{bobot kriteria} \times \text{skor}/4) \times 100$.

I. Keterkaitan dengan MK Pengembangan Aplikasi Web

- Repositori yang dibuat pada modul ini merupakan repositori produk yang digunakan sepanjang semester, termasuk untuk Tugas 1 dengan penanda versi v0.1 pada pekan 3.
- Pengamatan lalu lintas HTTP mendukung materi pekan 1 MK Pengembangan Aplikasi Web mengenai arsitektur aplikasi web dan protokol HTTP, serta Kuis 1.
- Praktikum menilai penguasaan teknik dan dokumentasi teknis, sedangkan fitur produk dan demonstrasi dinilai pada MK Pengembangan Aplikasi Web.

J. Troubleshooting

Masalah	Penyebab	Penyelesaian
'node' atau 'git' is not recognized	Variabel PATH belum diperbarui	Tutup lalu buka kembali terminal dan Visual Studio Code; apabila masih gagal, pasang ulang dengan opsi penambahan ke PATH
PowerShell: <i>running scripts is disabled on this system</i> saat menjalankan npx atau npm	Kebijakan eksekusi skrip PowerShell	Jalankan Set-ExecutionPolicy - Scope CurrentUser RemoteSigned, atau gunakan Command Prompt maupun Git Bash
Author identity unknown saat commit	Identitas Git belum diatur	Jalankan perintah git config pada Bagian D.2
Push ditolak dengan pesan autentikasi atau kode 403	Menggunakan kata sandi akun atau kredensial kedaluwarsa	Gunakan Git Credential Manager, gh auth login, atau kunci SSH
Updates were rejected because the remote contains work that you do not have locally	Repositori GitHub dibuat dengan README atau lisensi	Jalankan git pull --rebase origin main, lalu ulangi push
Port 3000 is in use	Server lain menggunakan port 3000	Hentikan proses tersebut atau gunakan port yang ditawarkan Next.js
Peringatan LF will be replaced by CRLF	Perbedaan akhir baris Windows dan Unix	Peringatan ini tidak menimbulkan galat; biarkan pengaturan bawaan Git for Windows
Keluaran curl di PowerShell berbeda dari modul	curl merupakan alias Invoke-WebRequest pada Windows PowerShell	Gunakan curl.exe
Terminal menampilkan editor Vim saat commit atau merge	Editor bawaan Git adalah Vim	Ketik :wq lalu Enter; atur core.editor sesuai Bagian D.2

K. Ketentuan Umum

- Asisten kecerdasan artifisial diperkenankan pada Tugas Pendahuluan dan Dokumen Teknis dengan kewajiban mencatat penggunaannya. Tes dikerjakan tanpa bantuan kecerdasan artifisial.
- Setiap mahasiswa wajib mampu menjelaskan setiap perintah dan kode yang dikumpulkan.

- Jangan memasukkan token, kata sandi, atau isi berkas `.env` ke layanan kecerdasan artifisial maupun ke repositori.
- Kendala teknis disampaikan melalui Forum Tanya Jawab pada e-learning dengan menyertakan sistem operasi, perintah yang dijalankan, dan pesan galat lengkap.

Daftar Pustaka

- [1] Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book>
- [2] Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP Semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- [3] Mozilla. (2026). An overview of HTTP. *MDN Web Docs*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- [4] Google. (2026). Inspect network activity. *Chrome for Developers*. <https://developer.chrome.com/docs/devtools/network>
- [5] Vercel. (2026). Installation. *Next.js Documentation*. <https://nextjs.org/docs/app/getting-started/installation>
- [6] GitHub. (2026). About authentication to GitHub. *GitHub Docs*. <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github>
- [7] GitHub. (2026). About *pull* requests. *GitHub Docs*. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- [8] Conventional Commits. (2019). *Conventional Commits 1.0.0*. <https://www.conventionalcommits.org/en/v1.0.0/>
- [9] OpenJS Foundation. (2026). Node.js releases. *Node.js*. <https://nodejs.org/en/about/previous-releases>
- [10] Stenberg, D. (2026). *curl man page*. <https://curl.se/docs/manpage.html>